Woxsen University

School of Technology

A PROJECT REPORT ON

DESIGN AND IMPLEMENTATION OF A DBMS FOR RECRUITMENT AND APPLICANT TRACKING SYSTEM

Submitted by:

Name- Mayank Panigrahi(25WU0102154)

Course-Database Management Systems

Year- 2025-2029

Branch- CSE-AIML

© School of Technology, Woxsen University, 2025. All rights reserved.

2. Abstract

The project, titled “Design and Implementation of a DBMS for Recruitment and Applicant Tracking System”, presents a relational database for organizing and managing the recruitment lifecycle. The system centralizes information related to departments, vacancies, candidates, applications, skills, qualifications, interview panels, interview rounds, schedules, evaluations, offers and joining records.

The main purpose of the database is to reduce scattered and duplicated recruitment information and provide a structured way to track a candidate from application through interview, evaluation, offer and joining. The database uses primary keys, foreign keys, composite keys and unique constraints to maintain relationships and data integrity. A web frontend is also used as a user-facing layer over the MySQL database through a backend/API.

The project presentation identifies the main problem as recruitment information being scattered across emails, spreadsheets and separate records. It proposes a centralized relational system connecting the complete recruitment lifecycle. This report follows that project scope and documents the implemented database, relational schema, SQL operations, queries, web interface, connectivity and testing.

Source basis: supplied project presentation, page 2, and the project SQL database script. The presentation explicitly identifies the scattered-data problem and centralized relational solution.

3. Introduction and Problem Statement

3.1 Introduction

Recruitment involves several connected activities such as creating vacancies, receiving applications, shortlisting candidates, conducting interviews, recording evaluations, issuing offers and maintaining joining information. Because these activities are related, a database is useful for maintaining a consistent record of the complete process.

3.2 Problem Statement

- Recruitment information may be scattered across emails, spreadsheets and separate records.
- Maintaining a complete history of each candidate can become difficult.
- Manual tracking can result in duplicate data, missed interviews and inconsistent application statuses.
- HR departments require a centralized system for vacancies, candidates, applications, interviews, evaluations, offers and joining records.
- A relational database is therefore required to connect the major stages of the recruitment lifecycle.

4. Objectives and Scope

4.1 Objectives

Centralize Recruitment Data: Store recruitment-related data under one database.

Reduce Data Redundancy: Normalize the data up to Third Normal Form (3NF).

Maintain Data Integrity: Use primary keys, foreign keys, UNIQUE constraints, NOT NULL constraints and DEFAULT values where applicable.

Track Candidate Progress: Follow candidates through application, interview, evaluation, offer and joining stages.

Support Reporting and Analysis: Use SQL queries, joins, aggregate functions, nested queries and views where required.

Provide a Foundation for a Front End: Provide a web interface connected to the backend/API and MySQL database.

4.2 Scope

Area	Scope
Vacancy Management	Create/manage vacancies, define positions and track vacancy status.
Candidate Management	Store candidate information, skills and qualifications.
Application Tracking	Record applications, track application status and support shortlisting/selection.
Interview Management	Create interview rounds, assign panels, schedule interviews and record evaluations.
Offer and Joining	Generate offers, track offer status and record joining information.
Reporting	Generate recruitment reports and analyze applications, interviews and hiring.

5. Software and Hardware Requirements

5.1 Software Requirements

Component	Requirement / Use
Database	MySQL
Database Management	MySQL Workbench / MySQL client
Frontend	HTML, CSS and JavaScript web frontend
Backend / API	Node.js with Express.js
Development Environment	Visual Studio Code
Version Control	Git and GitHub
Browser	Modern web browser

5.2 Hardware Requirements

- Laptop or desktop computer capable of running MySQL, the development environment and the web application.
- Minimum 4 GB RAM recommended; 8 GB or more is preferable for development.
- Stable internet connection for package installation, GitHub and related development tasks.
- No dedicated database server hardware is required for the academic implementation; MySQL runs locally.

6. ER Diagram

The ER model represents the major entities involved in the recruitment and applicant tracking process. The supplied diagram contains Department, Vacancy, Application, Candidate, Interview Panel, Interview Round,

Schedule, Qualification, Candidate Skill, Skill, Evaluation, Offer and Joining. The diagram uses rectangles for entities, ovals for attributes, diamonds for relationships and cardinalities such as 1:N and 0:1.

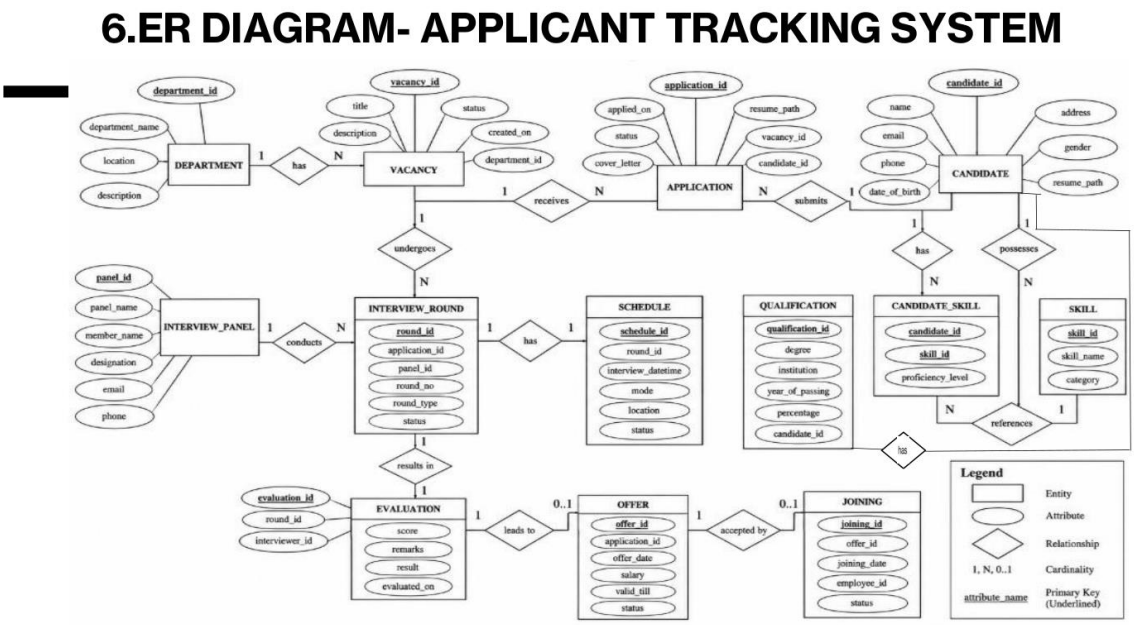


Figure 1. ER Diagram of the Applicant Tracking System (from supplied project presentation, page 8)

6.1 Main Relationships and Cardinalities

Relationship	Cardinality	Meaning
Department – Vacancy	1 : N	One department can have multiple vacancies; each vacancy belongs to a department.
Vacancy – Application	1 : N	One vacancy can receive multiple applications.
Candidate – Application	1 : N	One candidate can submit multiple applications.
Candidate – Qualification	1 : N	A candidate can have multiple qualification records.
Candidate – Candidate Skill	1 : N	A candidate can have multiple candidate-skill records.
Skill – Candidate Skill	1 : N	A skill can be associated with multiple candidates.
Interview Panel – Interview Round	1 : N	A panel can conduct multiple interview rounds.
Application – Interview Round	1 : N	An application can progress through multiple interview rounds.
Interview Round – Schedule	1 : 1	Each interview round has one schedule in the implemented schema.
Interview Round – Evaluation	1 : 1	Each interview round has one evaluation in the implemented schema.
Application – Offer	1 : N	An application can be associated with offer records; the project data uses selected applications for offers.
Offer – Joining	1 : 0..1	An offer may have one joining record, but

Relationship	Cardinality	Meaning
		joining is optional.

7. Relational Schema and Normalization

7. Relational Schema and Normalization

The relational schema is presented with each entity/relation in its own separate table. This directly represents the 13 tables implemented in the MySQL database.

1. DEPARTMENT

Attribute	Data Type	Key / Constraint
department_id	INT	PK, AUTO_INCREMENT
department_name	VARCHAR(100)	
location	VARCHAR(100)	
description	VARCHAR(255)	

2. CANDIDATE

Attribute	Data Type	Key / Constraint
candidate_id	INT	PK, AUTO_INCREMENT
name	VARCHAR(100)	
email	VARCHAR(100)	
phone	VARCHAR(20)	
date_of_birth	DATE	
address	VARCHAR(255)	
gender	VARCHAR(20)	
resume_path	VARCHAR(255)	

3. SKILL

Attribute	Data Type	Key / Constraint
skill_id	INT	PK, AUTO_INCREMENT
skill_name	VARCHAR(100)	
category	VARCHAR(100)	

4. INTERVIEW_PANEL

Attribute	Data Type	Key / Constraint
panel_id	INT	PK, AUTO_INCREMENT
panel_name	VARCHAR(100)	
member_name	VARCHAR(100)	
designation	VARCHAR(100)	
email	VARCHAR(100)	
phone	VARCHAR(20)	

5. VACANCY

Attribute	Data Type	Key / Constraint
vacancy_id	INT	PK, AUTO_INCREMENT
title	VARCHAR(150)	
description	VARCHAR(255)	
status	VARCHAR(50)	
created_on	DATE	
department_id	INT	FK → DEPARTMENT(department_id)

6. APPLICATION

Attribute	Data Type	Key / Constraint
application_id	INT	PK, AUTO_INCREMENT
applied_on	DATE	
status	VARCHAR(50)	
cover_letter	VARCHAR(255)	
resume_path	VARCHAR(255)	
vacancy_id	INT	FK → VACANCY(vacancy_id)
candidate_id	INT	FK → CANDIDATE(candidate_id)

7. QUALIFICATION

Attribute	Data Type	Key / Constraint
qualification_id	INT	PK, AUTO_INCREMENT
degree	VARCHAR(100)	
institution	VARCHAR(150)	
year_of_passing	INT	
percentage	DECIMAL(5,2)	
candidate_id	INT	FK → CANDIDATE(candidate_id)

8. CANDIDATE_SKILL

Attribute	Data Type	Key / Constraint
candidate_id	INT	PK + FK → CANDIDATE(candidate_id)
skill_id	INT	PK + FK → SKILL(skill_id)
proficiency_level	VARCHAR(50)	

9. INTERVIEW_ROUND

Attribute	Data Type	Key / Constraint
round_id	INT	PK, AUTO_INCREMENT
application_id	INT	FK → APPLICATION(application_id)
panel_id	INT	FK → INTERVIEW_PANEL(panel_id)
round_no	INT	

Attribute	Data Type	Key / Constraint
round_type	VARCHAR(100)	
status	VARCHAR(50)	

10. SCHEDULE

Attribute	Data Type	Key / Constraint
schedule_id	INT	PK, AUTO_INCREMENT
round_id	INT	UNIQUE, FK → INTERVIEW_ROUND(round_id)
interview_datetime	DATETIME	
mode	VARCHAR(50)	
location	VARCHAR(150)	
status	VARCHAR(50)	

11. EVALUATION

Attribute	Data Type	Key / Constraint
evaluation_id	INT	PK, AUTO_INCREMENT
round_id	INT	UNIQUE, FK → INTERVIEW_ROUND(round_id)
interviewer_id	INT	
score	DECIMAL(5,2)	
remarks	VARCHAR(255)	
result	VARCHAR(50)	
evaluated_on	DATE	

12. OFFER

Attribute	Data Type	Key / Constraint
offer_id	INT	PK, AUTO_INCREMENT
application_id	INT	FK → APPLICATION(application_id)
offer_date	DATE	
salary	DECIMAL(12,2)	
valid_till	DATE	
status	VARCHAR(50)	

13. JOINING

Attribute	Data Type	Key / Constraint
joining_id	INT	PK, AUTO_INCREMENT
offer_id	INT	UNIQUE, FK → OFFER(offer_id)
joining_date	DATE	
employee_id	INT	
status	VARCHAR(50)	

7.1 Normalization

The database is designed to satisfy Third Normal Form (3NF). The following table shows how normalization is applied to the complete schema.

Normal Form	Rule	Application to This Database	Result
1NF	All attributes contain atomic values; repeating groups are separated into individual records.	CANDIDATE_SKILL stores one candidate-skill combination per row.	Atomic values and no repeating groups.
2NF	Every non-key attribute depends on the complete primary key.	CANDIDATE_SKILL uses the composite key (candidate_id, skill_id), and proficiency_level depends on both attributes.	No partial dependency.
3NF	Non-key attributes depend only on the primary key and not on another non-key attribute.	Department, skill, candidate, vacancy and other independent information are stored in separate relations and connected using foreign keys.	Reduced redundancy and fewer update anomalies.

7.2 Entity-wise Normalization

Entity / Relation	Functional Dependency	Normalization Treatment	Normal Form
DEPARTMENT	department_id → department_name, location, description	No repeating attributes	3NF
CANDIDATE	candidate_id → candidate details	Candidate information stored once	3NF
SKILL	skill_id → skill_name, category	Skill details stored independently	3NF
INTERVIEW_PANEL	panel_id → panel details	Panel details stored independently	3NF
VACANCY	vacancy_id → vacancy details; department_id is a reference	Department data not repeated	3NF
APPLICATION	application_id → application details	Candidate/vacancy details referenced by FK	3NF
QUALIFICATION	qualification_id → qualification details	Candidate referenced by FK	3NF
CANDIDATE_SKILL	(candidate_id, skill_id) → proficiency_level	Composite key removes partial dependency	3NF
INTERVIEW_ROUND	round_id → round details	Application/panel referenced by FK	3NF
SCHEDULE	schedule_id → schedule details	Round referenced by unique FK	3NF
EVALUATION	evaluation_id → evaluation details	Round referenced by unique FK	3NF
OFFER	offer_id → offer details	Application referenced by FK	3NF
JOINING	joining_id → joining details	Offer referenced by unique FK	3NF

Thus, every entity is shown as a separate relation, and the complete schema is normalized through 3NF.

8. Data Dictionary

The following data dictionary is based on the actual SQL script supplied for the RecruitmentDB database.

Entity	Attribute	Data Type	Constraint	Description
DEPARTMENT	department_id	INT	PK, AUTO_INCREMENT	Unique department identifier
DEPARTMENT	department_name	VARCHAR(100)	—	Department name
DEPARTMENT	location	VARCHAR(100)	—	Department location
DEPARTMENT	description	VARCHAR(255)	—	Department description
CANDIDATE	candidate_id	INT	PK, AUTO_INCREMENT	Unique candidate identifier
CANDIDATE	name	VARCHAR(100)	—	Candidate name
CANDIDATE	email	VARCHAR(100)	—	Candidate email

Entity	Attribute	Data Type	Constraint	Description
CANDIDATE	phone	VARCHAR(20)	—	Candidate phone number
CANDIDATE	date_of_birth	DATE	—	Candidate date of birth
CANDIDATE	address	VARCHAR(255)	—	Candidate address
CANDIDATE	gender	VARCHAR(20)	—	Candidate gender
CANDIDATE	resume_path	VARCHAR(255)	—	Resume file path
SKILL	skill_id	INT	PK, AUTO_INCREMENT	Unique skill identifier
SKILL	skill_name	VARCHAR(100)	—	Skill name
SKILL	category	VARCHAR(100)	—	Skill category
INTERVIEW_PANEL	panel_id	INT	PK, AUTO_INCREMENT	Unique panel identifier
INTERVIEW_PANEL	panel_name	VARCHAR(100)	—	Panel name
INTERVIEW_PANEL	member_name	VARCHAR(100)	—	Panel member
INTERVIEW_PANEL	designation	VARCHAR(100)	—	Member designation
INTERVIEW_PANEL	email	VARCHAR(100)	—	Panel email
INTERVIEW_PANEL	phone	VARCHAR(20)	—	Panel phone
VACANCY	vacancy_id	INT	PK, AUTO_INCREMENT	Unique vacancy identifier
VACANCY	title	VARCHAR(150)	—	Job title
VACANCY	description	VARCHAR(255)	—	Vacancy description
VACANCY	status	VARCHAR(50)	—	Vacancy status
VACANCY	created_on	DATE	—	Vacancy creation date
VACANCY	department_id	INT	FK	Department reference
APPLICATION	application_id	INT	PK, AUTO_INCREMENT	Unique application identifier
APPLICATION	applied_on	DATE	—	Application date
APPLICATION	status	VARCHAR(50)	—	Application status
APPLICATION	cover_letter	VARCHAR(255)	—	Application cover letter
APPLICATION	resume_path	VARCHAR(255)	—	Resume path stored for application
APPLICATION	vacancy_id	INT	FK	Vacancy reference
APPLICATION	candidate_id	INT	FK	Candidate reference
QUALIFICATION	qualification_id	INT	PK, AUTO_INCREMENT	Qualification identifier
QUALIFICATION	degree	VARCHAR(100)	—	Degree/qualification
QUALIFICATION	institution	VARCHAR(150)	—	Institution
QUALIFICATION	year_of_passing	INT	—	Year of passing
QUALIFICATION	percentage	DECIMAL(5,2)	—	Percentage/score
QUALIFICATION	candidate_id	INT	FK	Candidate reference
CANDIDATE_SKILL	candidate_id	INT	PK part, FK	Candidate reference
CANDIDATE_SKILL	skill_id	INT	PK part, FK	Skill reference
CANDIDATE_SKILL	proficiency_level	VARCHAR(50)	—	Candidate skill proficiency
INTERVIEW_ROUND	round_id	INT	PK, AUTO_INCREMENT	Round identifier
INTERVIEW_ROUND	application_id	INT	FK	Application reference
INTERVIEW_ROUND	panel_id	INT	FK	Panel reference
INTERVIEW_ROUND	round_no	INT	—	Interview round number
INTERVIEW_ROUND	round_type	VARCHAR(100)	—	Technical/HR etc.
INTERVIEW_ROUND	status	VARCHAR(50)	—	Round status

Entity	Attribute	Data Type	Constraint	Description
SCHEDULE	schedule_id	INT	PK, AUTO_INCREMENT	Schedule identifier
SCHEDULE	round_id	INT	UNIQUE, FK	Interview round reference
SCHEDULE	interview_datetime	DATETIME	—	Interview date and time
SCHEDULE	mode	VARCHAR(50)	—	Online/In-Person etc.
SCHEDULE	location	VARCHAR(150)	—	Interview location
SCHEDULE	status	VARCHAR(50)	—	Schedule status
EVALUATION	evaluation_id	INT	PK, AUTO_INCREMENT	Evaluation identifier
EVALUATION	round_id	INT	UNIQUE, FK	Interview round reference
EVALUATION	interviewer_id	INT	—	Interviewer identifier used in the data model
EVALUATION	score	DECIMAL(5,2)	—	Evaluation score
EVALUATION	remarks	VARCHAR(255)	—	Evaluator remarks
EVALUATION	result	VARCHAR(50)	—	Pass/result
EVALUATION	evaluated_on	DATE	—	Evaluation date
OFFER	offer_id	INT	PK, AUTO_INCREMENT	Offer identifier
OFFER	application_id	INT	FK	Application reference
OFFER	offer_date	DATE	—	Offer date
OFFER	salary	DECIMAL(12,2)	—	Offered salary
OFFER	valid_till	DATE	—	Offer validity date
OFFER	status	VARCHAR(50)	—	Offer status
JOINING	joining_id	INT	PK, AUTO_INCREMENT	Joining identifier
JOINING	offer_id	INT	UNIQUE, FK	Offer reference
JOINING	joining_date	DATE	—	Joining date
JOINING	employee_id	INT	—	Employee identifier
JOINING	status	VARCHAR(50)	—	Joining status

9. SQL Commands Used (DDL, DML) with Sample Outputs

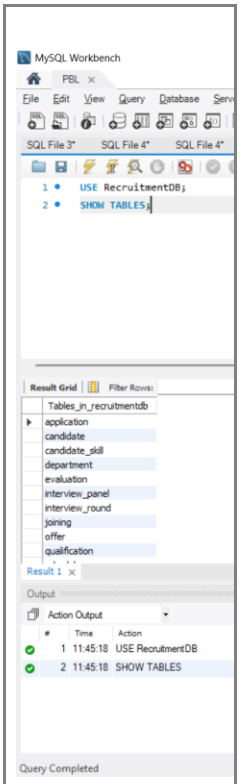
9.1 DDL Commands

DDL is used to create and define the database structure. The project SQL script creates the RecruitmentDB database and 13 related tables.

```
DROP DATABASE IF EXISTS RecruitmentDB;
CREATE DATABASE RecruitmentDB;
USE RecruitmentDB;

CREATE TABLE DEPARTMENT (
    department_id INT PRIMARY KEY AUTO_INCREMENT,
    department_name VARCHAR(100),
    location VARCHAR(100),
    description VARCHAR(255)
);
```

[SAMPLE DDL EXECUTION / SHOW TABLES OUTPUT]



Successful execution of the SHOW TABLES command displaying the 13 tables in the RecruitmentDB database.

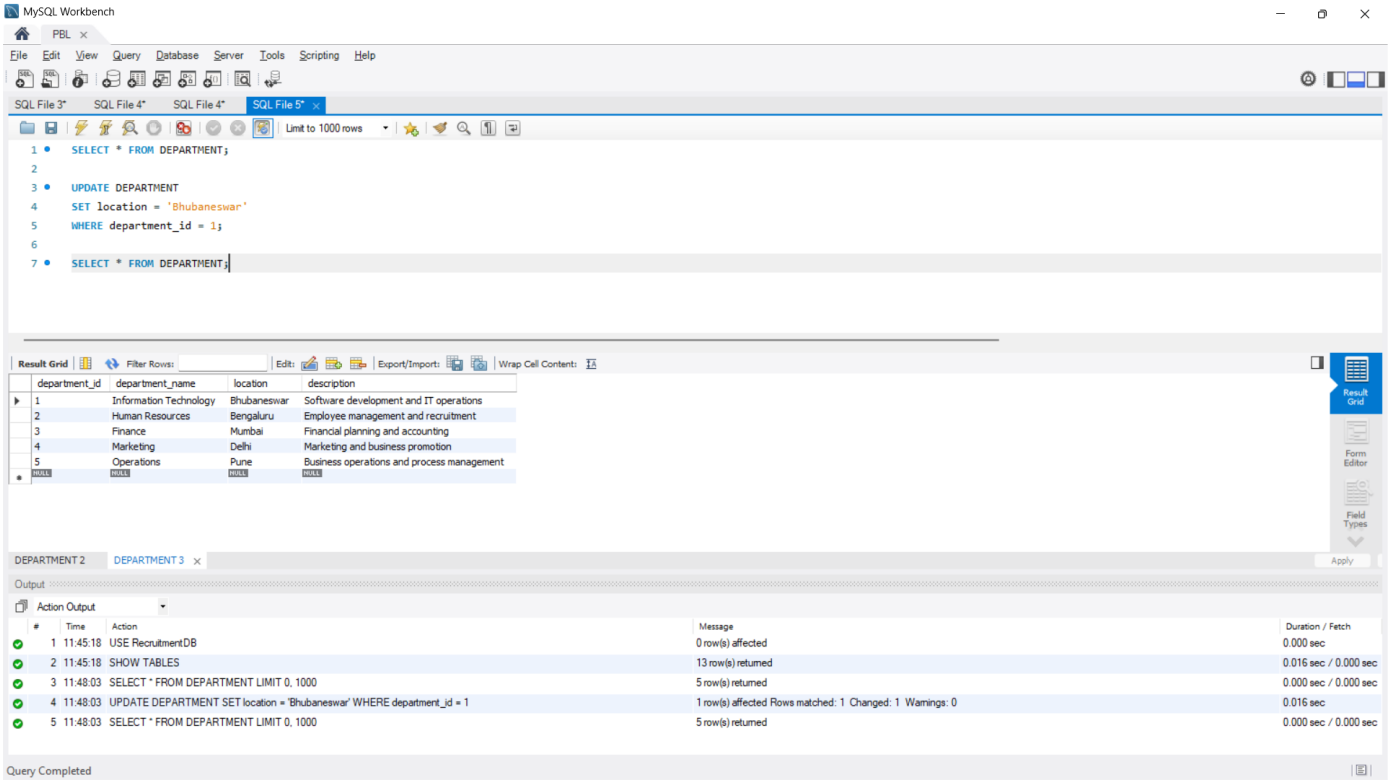
9.2 DML Commands

DML is used to insert and modify the records stored in the tables.

```
INSERT INTO DEPARTMENT
(department_id, department_name, location, description)
VALUES
(1, 'Information Technology', 'Hyderabad',
 'Software development and IT operations');

UPDATE DEPARTMENT
SET location = 'Bhubaneswar'
```

```
WHERE department_id = 1;
```



Execution of INSERT, UPDATE and SELECT operations demonstrating data manipulation in the RecruitmentDB database.

9.3 Constraints Used

- PRIMARY KEY: uniquely identifies records.
- FOREIGN KEY: maintains relationships between related tables.
- AUTO_INCREMENT: generates identifiers automatically for major entities.
- UNIQUE: used for one-to-one style relationships such as SCHEDULE.round_id, EVALUATION.round_id and JOINING.offer_id.
- Composite PRIMARY KEY: used in CANDIDATE_SKILL(candidate_id, skill_id).

10. Queries with Outputs

This section contains important queries for recruitment reporting, including the query used for the Presentation-II requirement. Actual MySQL result screenshots should be inserted in the reserved spaces.

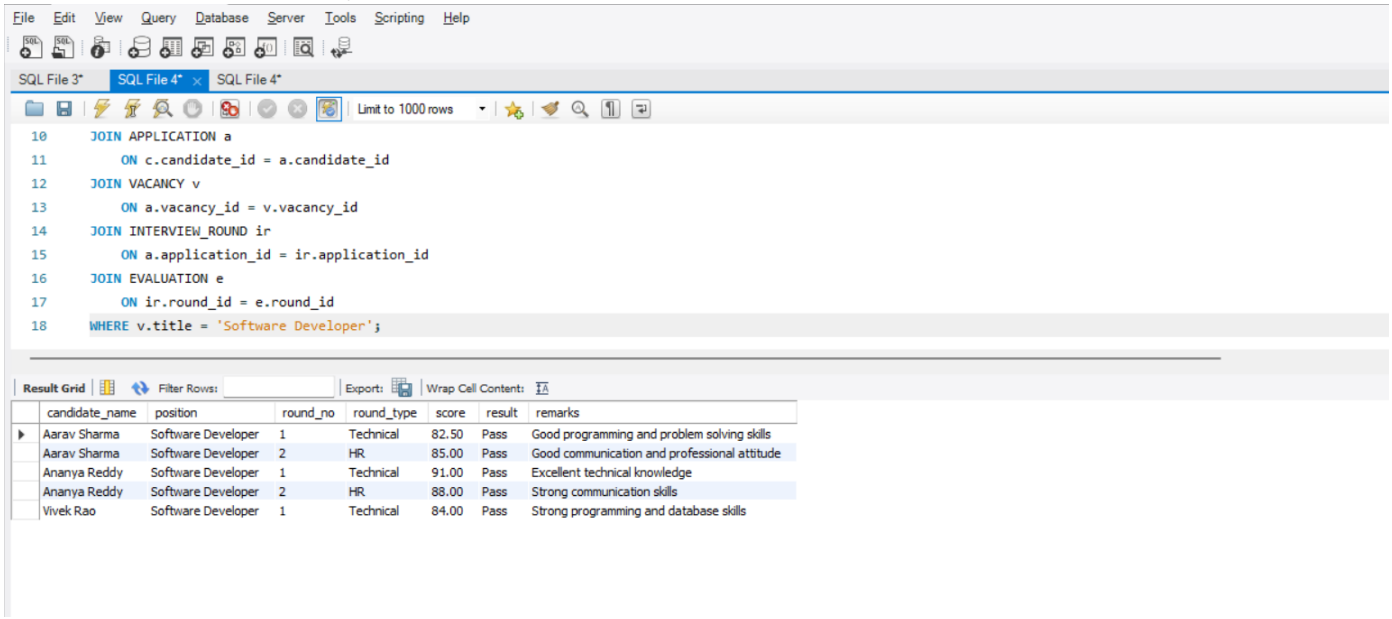
10.1 Interview Results for Software Developer(Presentation-II Query)

Purpose: Find interview results for candidates applying for the Software Developer vacancy.

```
SELECT c.name AS candidate_name,
       v.title AS vacancy,
       ir.round_no,
       e.score,
       e.result,
       e.remarks
```

```
FROM CANDIDATE c
JOIN APPLICATION a ON c.candidate_id = a.candidate_id
JOIN VACANCY v ON a.vacancy_id = v.vacancy_id
JOIN INTERVIEW_ROUND ir ON a.application_id = ir.application_id
JOIN EVALUATION e ON ir.round_id = e.round_id
WHERE v.title = 'Software Developer';
```

query-Find results of all interviews for software developer position



This query retrieves interview details for candidates who applied for the Software Developer vacancy. It combines candidate, application, vacancy, interview round and evaluation records using JOIN operations. The output displays candidate names, interview rounds, scores, results and interviewer remarks. This helps the recruitment team review candidate performance.

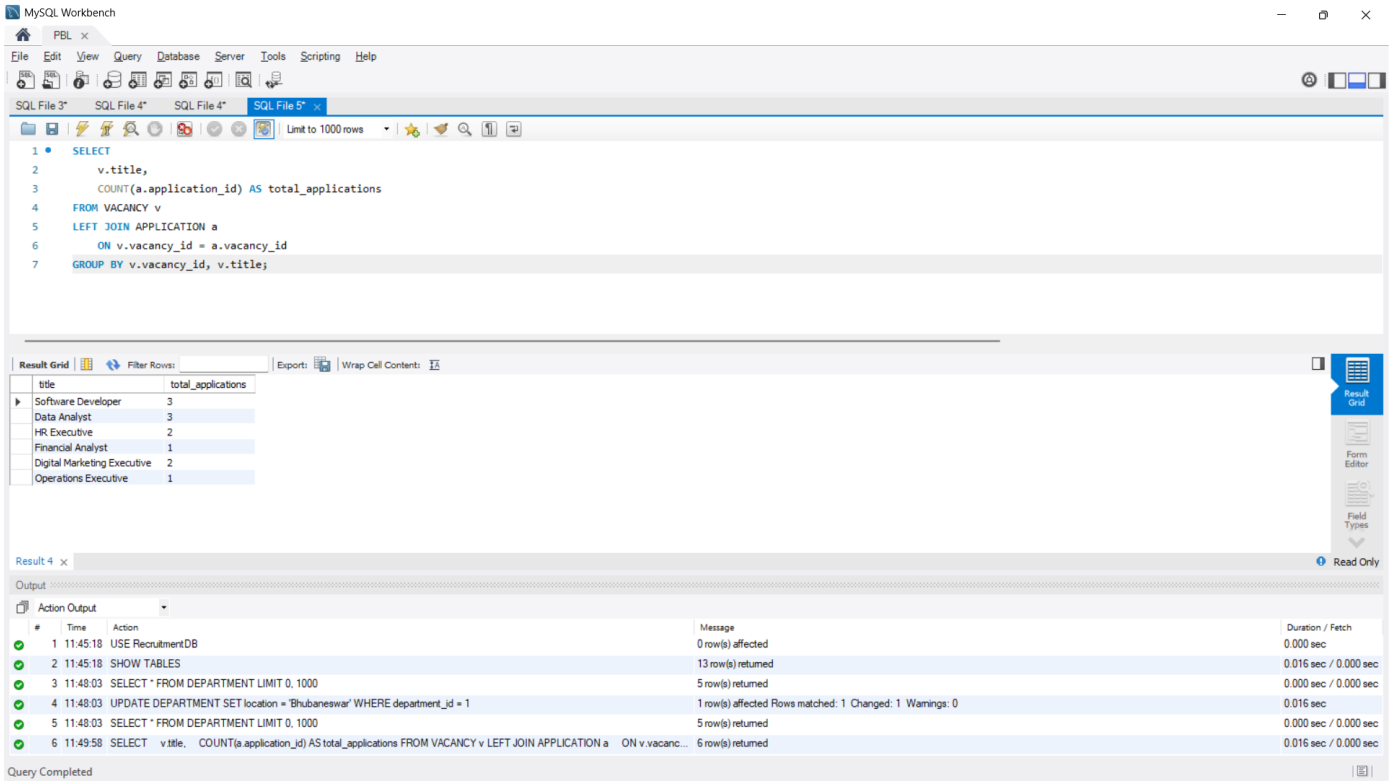
The actual number of rows and the interview results are based on the records stored in the database.



10.2 Applications by Vacancy

Purpose: Count the number of applications received by each vacancy.

```
SELECT v.title,
       COUNT(a.application_id) AS total_applications
FROM VACANCY v
LEFT JOIN APPLICATION a
       ON v.vacancy_id = a.vacancy_id
GROUP BY v.vacancy_id, v.title;
```

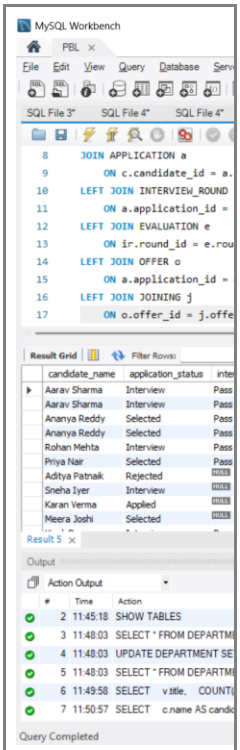

MySQL Workbench
PBL
File Edit View Query Database Server Tools Scripting Help
SQL File 3* SQL File 4* SQL File 4* SQL File 5*
Limit to 1000 rows
SELECT
v.title,
COUNT(a.application_id) AS total_applications
FROM VACANCY v
LEFT JOIN APPLICATION a
ON v.vacancy_id = a.vacancy_id
GROUP BY v.vacancy_id, v.title;
Result Grid
Filter Rows:
Export:
Wrap Cell Content:
title total_applications
Software Developer 3
Data Analyst 3
HR Executive 2
Financial Analyst 1
Digital Marketing Executive 2
Operations Executive 1
Result 4
Read Only
Output
Action Output
Time Action Message Duration / Fetch
1 11:45:18 USE RecruitmentDB 0 row(s) affected 0.000 sec
2 11:45:18 SHOW TABLES 13 row(s) returned 0.016 sec / 0.000 sec
3 11:48:03 SELECT * FROM DEPARTMENT LIMIT 0, 1000 5 row(s) returned 0.000 sec / 0.000 sec
4 11:48:03 UPDATE DEPARTMENT SET location = 'Bhubaneswar' WHERE department_id = 1 1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0 0.016 sec
5 11:48:03 SELECT * FROM DEPARTMENT LIMIT 0, 1000 5 row(s) returned 0.000 sec / 0.000 sec
6 11:49:58 SELECT v.title, COUNT(a.application_id) AS total_applications FROM VACANCY v LEFT JOIN APPLICATION a ON v.vacanc... 6 row(s) returned 0.016 sec / 0.000 sec
Query Completed

This query calculates the number of applications received for each vacancy. The LEFT JOIN includes vacancies even when no applications have been recorded. The COUNT function calculates the total applications, while GROUP BY groups the results by vacancy. This report helps compare the number of applications received for different job positions.

10.3 Candidate Progress

Purpose: Track a candidate from application status through interview result, offer and joining.

```
SELECT c.name,
       a.status AS application_status,
       e.result AS interview_result,
       o.status AS offer_status,
       j.status AS joining_status
FROM CANDIDATE c
JOIN APPLICATION a
     ON c.candidate_id = a.candidate_id
LEFT JOIN INTERVIEW_ROUND ir
     ON a.application_id = ir.application_id
LEFT JOIN EVALUATION e
     ON ir.round_id = e.round_id
LEFT JOIN OFFER o
     ON a.application_id = o.application_id
LEFT JOIN JOINING j
     ON o.offer_id = j.offer_id;
```



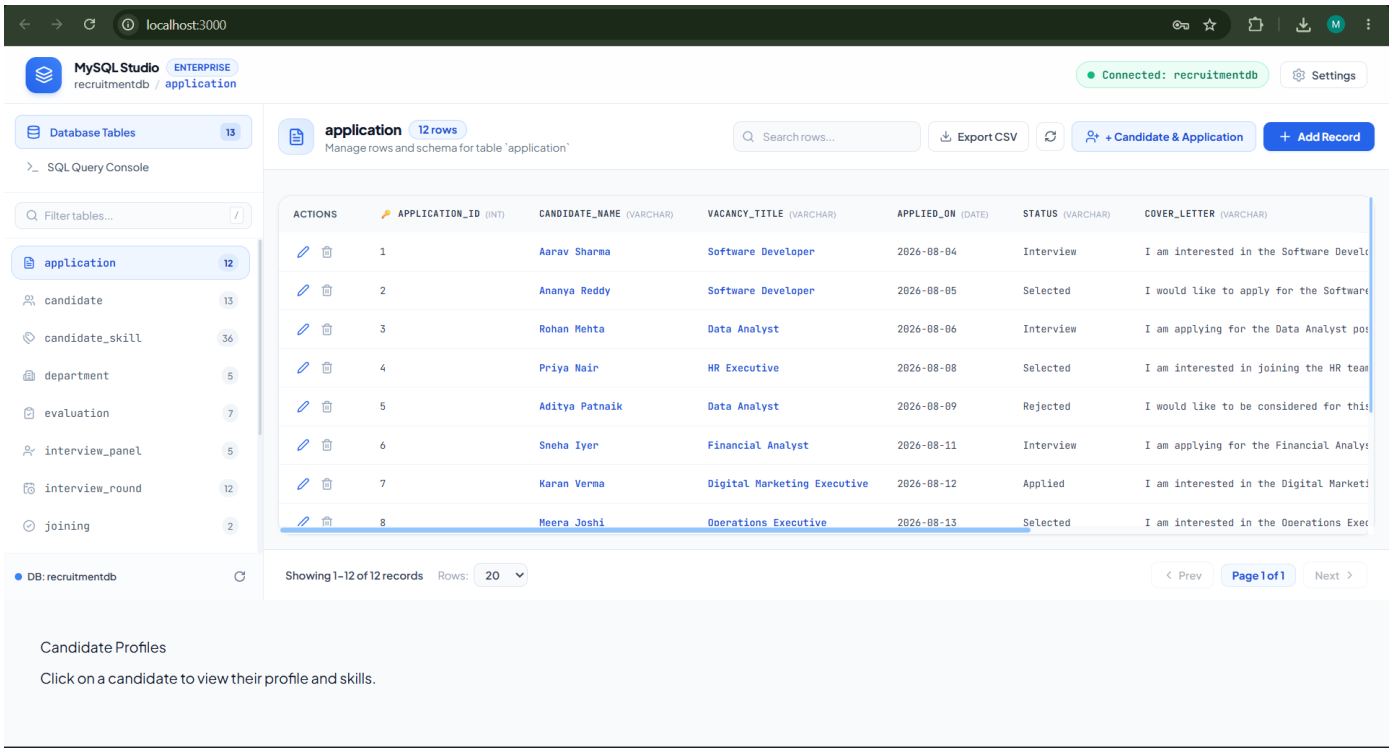
This query tracks candidates across the recruitment process, from application status to interview results, offer status and joining status. LEFT JOIN operations allow candidate applications to appear even when evaluation, offer or joining records are not yet available. The query demonstrates how related tables can be combined to provide a consolidated view of candidate progress.

10.4 Results Summary

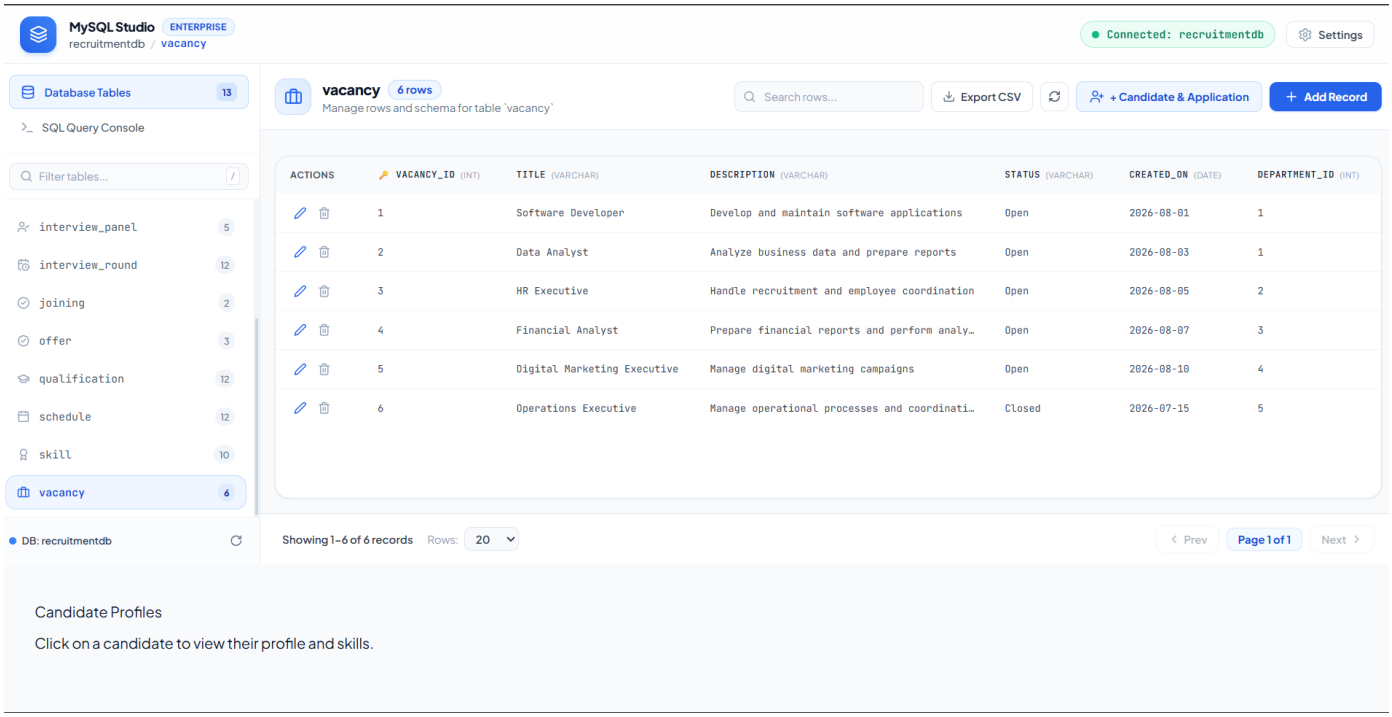
Query No.	Query Purpose	Actual Output / Finding
Q1	Interview results for Software Developer	Candidate interview results for the Software Developer vacancy, including round number, score, result and remarks.
Q2	Applications by vacancy	Application count for each vacancy, including vacancies with zero applications.
Q3	Candidate progress	Candidate application status, interview result, offer status and joining status.
Q4	Additional Presentation-II query	The actual result of the additional query required by your Presentation-II assignment.

11. UI Design and Screenshots

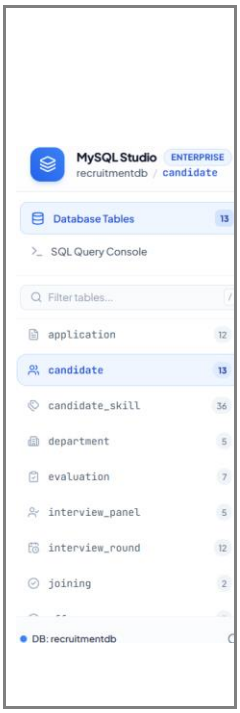
The web frontend provides a user-facing layer over the recruitment database. The following spaces are reserved for screenshots of the actual implemented interface.



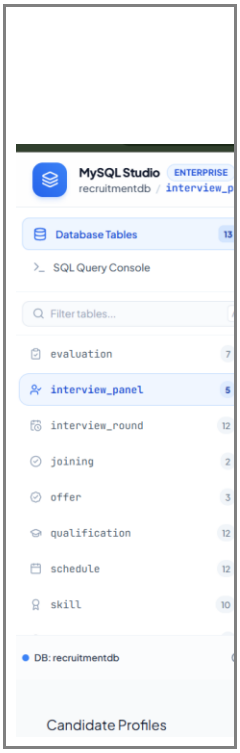
[VACANCY MANAGEMENT SCREENSHOT]



[CANDIDATE / APPLICATION SCREENSHOT]



[INTERVIEW / EVALUATION SCREENSHOT]


MySQL Studio
ENTERPRISE
recruitmentdb / interview_p
Database Tables
13
SQL Query Console
Filter tables...
evaluation
7
interview_panel
5
interview_round
12
joining
2
offer
3
qualification
12
schedule
12
skill
10
DB: recruitmentdb
Candidate Profiles


MySQL Studio
ENTERPRISE
recruitmentdb / offer
Database Tables
13
SQL Query Console
Filter tables...
evaluation
7
interview_panel
5
interview_round
12
joining
2
offer
3
qualification
12
schedule
12
skill
10
DB: recruitmentdb

12. Implementation Details

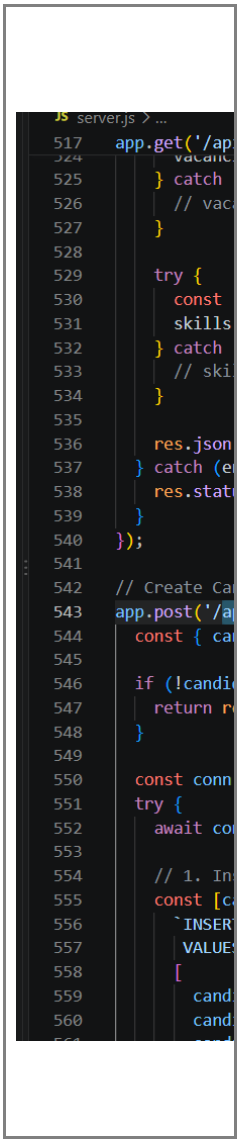
12.1 Technology Stack

Layer	Technology	Purpose
Database	MySQL	Stores and manages recruitment data.
Frontend	HTML, CSS, JavaScript	Provides the user interface.
Backend	Node.js + Express.js	Handles API requests and communicates with MySQL.
Development	Visual Studio Code	Development and code editing.
Version Control	Git / GitHub	Source-code management and repository hosting.

12.2 Database Connectivity

The implemented architecture follows the flow: Web Frontend → Backend/API → MySQL Database. The frontend sends HTTP requests to the backend. The backend executes SQL operations on MySQL and returns the database results to the frontend. This keeps database credentials on the server side and allows the website to display data stored in the actual MySQL database.

[BACKEND DATABASE CONNECTION CODE SCREENSHOT



[FRONTEND ↔ BACKEND / API SCREENSHOT]



12.3 Frontend, Backend and Database Workflow

12.3 Frontend, Backend and Database Workflow

The Recruitment and Applicant Tracking System follows a three-layer architecture consisting of the frontend, backend and database.

The frontend is developed using HTML, CSS and JavaScript. It provides the user interface through which users view recruitment information and interact with the system.

The backend uses Node.js and Express.js to handle requests from the frontend. It processes these requests, performs the required database operations and returns the results to the frontend.

MySQL acts as the database layer and stores information related to candidates, vacancies, applications, interview rounds, evaluations, offers and joining records.

The overall workflow is:

Frontend → Backend/API → MySQL Database → Backend/API → Frontend

This architecture separates the user interface from database operations and allows the website to retrieve and display recruitment information through the backend.

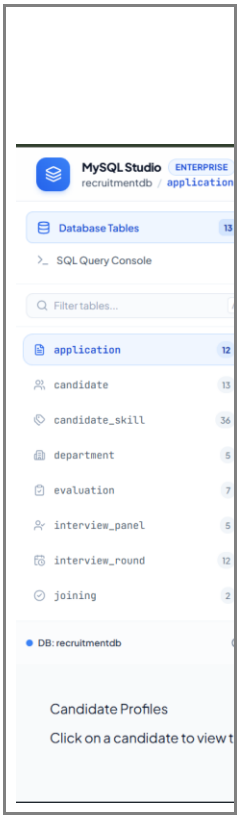
12.4 Project Structure

The project is organized into separate components for the frontend, backend and database. The frontend contains the files responsible for the user interface and styling. The backend contains the server-side code responsible for processing requests and communicating with MySQL. The database folder contains the SQL script used to create the recruitment tables and relationships. This separation makes the project easier to understand, test and maintain.



13. Testing (Test Cases and Results)

Test Case	Test	Expected Result	Actual Result	Status
TC01	Open website	Dashboard/page loads	Open	PASS
TC02	View vacancies	Vacancy records displayed	Shows vacancies	PASS
TC03	Search/view candidate	Correct candidate data displayed	Correct data displayed	PASS
TC04	Run Software Developer query	Interview results displayed	Correct results shown	PASS
TC05	Invalid/non-existing record	System handles no-result case	Works completely accrding to database	PASS
TC06	Database connectivity	Frontend/backend communicates with MySQL	Both are connected properly	PASS
TC07	Constraint validation	Invalid duplicate/missing data is rejected where constraints apply	Constraints work properly	PASS



14. Conclusion and Future Enhancements

14.1 Conclusion

The Recruitment and Applicant Tracking System provides a structured relational database for managing the main stages of recruitment. The ER model separates the major entities and connects them through meaningful relationships. The relational schema supports candidate, vacancy, application, interview, evaluation, offer and joining records, while normalization reduces unnecessary redundancy.

The project also demonstrates how SQL can be used to store data and generate useful recruitment information through joins, filtering and aggregation. The MySQL database serves as the data layer for the web application, allowing the frontend to display recruitment information through the backend/API.

14.2 Future Enhancements

- Role-based authentication for HR, interview panel members and administrators.
- Advanced search and filtering for candidates and vacancies.
- Dashboard charts for applications, interviews, offers and joining statistics.
- Automated email notifications for interview schedules and offer status.
- Resume upload and document management.
- Audit logs to record important changes to recruitment records.
- Deployment of the database/API and web application to a secure hosted environment.

15. References

- Supplied project presentation: “Design and Implementation of a DBMS for Recruitment and Applicant Tracking System.”

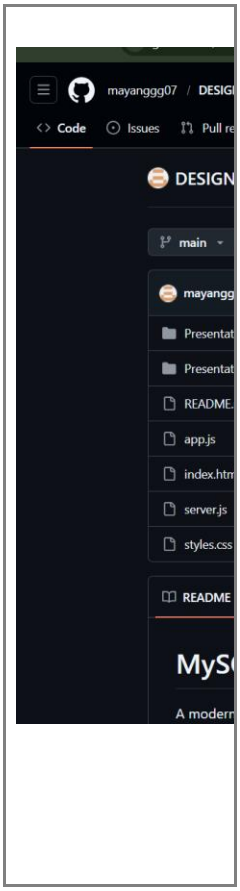
- MySQL Documentation, for SQL syntax, database creation, constraints, joins and query operations.
- DBMS course notes and laboratory material provided by the faculty.
- Project SQL database script: RecruitmentDB.

16. Appendix: GitHub Repository Link

GitHub Repository: https://github.com/mayanggg07/DESIGN-AND-IMPLEMENTATION-OF-A-DBMS-FOR-RECRUITMENT-AND-APPLICANT-TRACKING-SYSTEM

The repository should contain the frontend source code, backend/API source code, SQL script, README and supporting project files.

[



Appendix A: Relational Schema Quick Reference

- DEPARTMENT(department_id PK, department_name, location, description)
- CANDIDATE(candidate_id PK, name, email, phone, date_of_birth, address, gender, resume_path)
- SKILL(skill_id PK, skill_name, category)
- INTERVIEW_PANEL(panel_id PK, panel_name, member_name, designation, email, phone)
- VACANCY(vacancy_id PK, title, description, status, created_on, department_id FK → DEPARTMENT.department_id)
- APPLICATION(application_id PK, applied_on, status, cover_letter, resume_path, vacancy_id FK → VACANCY.vacancy_id, candidate_id FK → CANDIDATE.candidate_id)

- QUALIFICATION(qualification_id PK, degree, institution, year_of_passing, percentage, candidate_id FK → CANDIDATE.candidate_id)
- CANDIDATE_SKILL(candidate_id FK → CANDIDATE.candidate_id, skill_id FK → SKILL.skill_id, proficiency_level, PK(candidate_id, skill_id))
- INTERVIEW_ROUND(round_id PK, application_id FK → APPLICATION.application_id, panel_id FK → INTERVIEW_PANEL.panel_id, round_no, round_type, status)
- SCHEDULE(schedule_id PK, round_id FK UNIQUE → INTERVIEW_ROUND.round_id, interview_datetime, mode, location, status)
- EVALUATION(evaluation_id PK, round_id FK UNIQUE → INTERVIEW_ROUND.round_id, interviewer_id, score, remarks, result, evaluated_on)
- OFFER(offer_id PK, application_id FK → APPLICATION.application_id, offer_date, salary, valid_till, status)
- JOINING(joining_id PK, offer_id FK UNIQUE → OFFER.offer_id, joining_date, employee_id, status)

Appendix B: SQL Script

```
-- ========================================================== -- RECRUITMENT DATABASE -- ==========================================================

DROP DATABASE IF EXISTS RecruitmentDB;

CREATE DATABASE RecruitmentDB;

USE RecruitmentDB;

-- ========================================================== -- 1. DEPARTMENT -- ==========================================================

CREATE TABLE DEPARTMENT ( department_id INT PRIMARY KEY AUTO_INCREMENT, department_name VARCHAR(100), location VARCHAR(100), description VARCHAR(255) );

-- ========================================================== -- 2. CANDIDATE -- ==========================================================

CREATE TABLE CANDIDATE ( candidate_id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(100), email VARCHAR(100), phone VARCHAR(20), date_of_birth DATE, address VARCHAR(255), gender VARCHAR(20), resume_path VARCHAR(255) );

-- ========================================================== -- 3. SKILL -- ==========================================================

CREATE TABLE SKILL ( skill_id INT PRIMARY KEY AUTO_INCREMENT, skill_name VARCHAR(100), category VARCHAR(100) );

-- ========================================================== -- 4. INTERVIEW_PANEL -- ==========================================================

CREATE TABLE INTERVIEW_PANEL ( panel_id INT PRIMARY KEY AUTO_INCREMENT, panel_name VARCHAR(100), member_name VARCHAR(100), designation VARCHAR(100), email VARCHAR(100), phone VARCHAR(20) );
```

-- ========== -- 5. VACANCY -- DEPARTMENT 1 : N VACANCY -- ==========

CREATE TABLE VACANCY (vacancy_id INT PRIMARY KEY AUTO_INCREMENT, title VARCHAR(150), description VARCHAR(255), status VARCHAR(50), created_on DATE, department_id INT,

```
FOREIGN KEY (department_id)
    REFERENCES DEPARTMENT(department_id)
```

);

-- ========== -- 6. APPLICATION -- VACANCY 1 : N APPLICATION -- CANDIDATE 1 : N APPLICATION -- ==========

CREATE TABLE APPLICATION (application_id INT PRIMARY KEY AUTO_INCREMENT, applied_on DATE, status VARCHAR(50), cover_letter VARCHAR(255), resume_path VARCHAR(255), vacancy_id INT, candidate_id INT,

```
FOREIGN KEY (vacancy_id)
    REFERENCES VACANCY(vacancy_id),

FOREIGN KEY (candidate_id)
    REFERENCES CANDIDATE(candidate_id)
```

);

-- ========== -- 7. QUALIFICATION -- ==========

CREATE TABLE QUALIFICATION (qualification_id INT PRIMARY KEY AUTO_INCREMENT, degree VARCHAR(100), institution VARCHAR(150), year_of_passing INT, percentage DECIMAL(5,2), candidate_id INT,

```
FOREIGN KEY (candidate_id)
    REFERENCES CANDIDATE(candidate_id)
```

);

-- ========== -- 8. CANDIDATE_SKILL -- CANDIDATE 1 : N CANDIDATE_SKILL -- SKILL 1 : N CANDIDATE_SKILL -- ==========

CREATE TABLE CANDIDATE_SKILL (candidate_id INT, skill_id INT, proficiency_level VARCHAR(50),

```
PRIMARY KEY (candidate_id, skill_id),
```

```
FOREIGN KEY (candidate_id)
    REFERENCES CANDIDATE(candidate_id),

FOREIGN KEY (skill_id)
    REFERENCES SKILL(skill_id)


);

-- ========================================================= -- 9. INTERVIEW_ROUND --
INTERVIEW_PANEL 1 : N INTERVIEW_ROUND --
=========================================================

CREATE TABLE INTERVIEW_ROUND ( round_id INT PRIMARY KEY AUTO_INCREMENT, application_id
INT, panel_id INT, round_no INT, round_type VARCHAR(100), status VARCHAR(50),

FOREIGN KEY (application_id)
    REFERENCES APPLICATION(application_id),

FOREIGN KEY (panel_id)
    REFERENCES INTERVIEW_PANEL(panel_id)


);

-- ========================================================= -- 10. SCHEDULE --
INTERVIEW_ROUND 1 : 1 SCHEDULE --
=========================================================

CREATE TABLE SCHEDULE ( schedule_id INT PRIMARY KEY AUTO_INCREMENT, round_id INT UNIQUE,
interview_datetime DATETIME, mode VARCHAR(50), location VARCHAR(150), status VARCHAR(50),

FOREIGN KEY (round_id)
    REFERENCES INTERVIEW_ROUND(round_id)


);

-- ========================================================= -- 11. EVALUATION --
INTERVIEW_ROUND 1 : 1 EVALUATION --
=========================================================

CREATE TABLE EVALUATION ( evaluation_id INT PRIMARY KEY AUTO_INCREMENT, round_id INT
UNIQUE, interviewer_id INT, score DECIMAL(5,2), remarks VARCHAR(255), result VARCHAR(50),
evaluated_on DATE,

FOREIGN KEY (round_id)
    REFERENCES INTERVIEW_ROUND(round_id)
```

);

-- ========== -- 12. OFFER -- ==========

CREATE TABLE OFFER (offer_id INT PRIMARY KEY AUTO_INCREMENT, application_id INT, offer_date DATE, salary DECIMAL(12,2), valid_till DATE, status VARCHAR(50),

```
FOREIGN KEY (application_id)
    REFERENCES APPLICATION(application_id)
```

);

-- ========== -- 13. JOINING -- OFFER 1 : 0..1 JOINING -- ==========

CREATE TABLE JOINING (joining_id INT PRIMARY KEY AUTO_INCREMENT, offer_id INT UNIQUE, joining_date DATE, employee_id INT, status VARCHAR(50),

```
FOREIGN KEY (offer_id)
    REFERENCES OFFER(offer_id)
```

);

-- ========== -- INSERT DEPARTMENT DATA -- ==========

INSERT INTO DEPARTMENT (department_id, department_name, location, description) VALUES (1, 'Information Technology', 'Hyderabad', 'Software development and IT operations'),

(2, 'Human Resources', 'Bengaluru', 'Employee management and recruitment'),

(3, 'Finance', 'Mumbai', 'Financial planning and accounting'),

(4, 'Marketing', 'Delhi', 'Marketing and business promotion'),

(5, 'Operations', 'Pune', 'Business operations and process management');

-- ========== -- INSERT CANDIDATE DATA -- ==========

INSERT INTO CANDIDATE (candidate_id, name, email, phone, date_of_birth, address, gender, resume_path) VALUES

(1, 'Aarav Sharma', 'aarav.sharma@gmail.com', '9876543210', '2001-05-14', 'Kukatpally, Hyderabad', 'Male', '/resumes/aarav_sharma.pdf'),

(2, 'Ananya Reddy', 'ananya.reddy@gmail.com', '9876543211', '2002-08-21', 'Madhapur, Hyderabad', 'Female', '/resumes/ananya_reddy.pdf'),

(3, 'Rohan Mehta', 'rohan.mehta@gmail.com', '9876543212', '2000-11-09', 'Andheri, Mumbai', 'Male', '/resumes/rohan_mehta.pdf'),

(4, 'Priya Nair', 'priya.nair@gmail.com', '9876543213', '2001-03-17', 'Koramangala, Bengaluru', 'Female', '/resumes/priya_nair.pdf'),

(5, 'Aditya Patnaik', 'aditya.patnaik@gmail.com', '9876543214', '2002-01-25', 'Baramunda, Bhubaneswar', 'Male', '/resumes/aditya_patnaik.pdf'),

(6, 'Sneha Iyer', 'sneha.iyer@gmail.com', '9876543215', '2001-07-30', 'Baner, Pune', 'Female', '/resumes/sneha_iyer.pdf'),

(7, 'Karan Verma', 'karan.verma@gmail.com', '9876543216', '2000-12-11', 'Noida, Uttar Pradesh', 'Male', '/resumes/karan_verma.pdf'),

(8, 'Meera Joshi', 'meera.joshi@gmail.com', '9876543217', '2002-04-06', 'Jaipur, Rajasthan', 'Female', '/resumes/meera_joshi.pdf'),

(9, 'Vivek Rao', 'vivek.rao@gmail.com', '9876543218', '2001-09-19', 'Secunderabad, Hyderabad', 'Male', '/resumes/vivek_rao.pdf'),

(10, 'Ishita Kapoor', 'ishita.kapoor@gmail.com', '9876543219', '2002-02-12', 'Gurugram, Haryana', 'Female', '/resumes/ishita_kapoor.pdf'),

(11, 'Rahul Das', 'rahul.das@gmail.com', '9876543220', '2000-06-28', 'Salt Lake, Kolkata', 'Male', '/resumes/rahul_das.pdf'),

(12, 'Kavya Menon', 'kavya.menon@gmail.com', '9876543221', '2001-10-05', 'Kochi, Kerala', 'Female', '/resumes/kavya_menon.pdf');

-- ========== -- INSERT SKILL DATA -- ==========

INSERT INTO SKILL (skill_id, skill_name, category) VALUES

(1, 'Python', 'Programming'), (2, 'Java', 'Programming'), (3, 'SQL', 'Database'), (4, 'Machine Learning', 'Artificial Intelligence'), (5, 'Data Analysis', 'Data Science'), (6, 'JavaScript', 'Web Development'), (7, 'Communication', 'Soft Skill'), (8, 'Project Management', 'Management'), (9, 'Digital Marketing', 'Marketing'), (10, 'Financial Analysis', 'Finance');

-- ========== -- INSERT INTERVIEW PANEL DATA -- ==========

INSERT INTO INTERVIEW_PANEL (panel_id, panel_name, member_name, designation, email, phone) VALUES

(1, 'Technical Panel A', 'Rahul Menon', 'Senior Software Engineer', 'rahul.menon@company.com', '9000000001'),

(2, 'Technical Panel B', 'Vikram Singh', 'Engineering Manager', 'vikram.singh@company.com', '9000000002'),

(3, 'HR Panel A', 'Neha Kapoor', 'HR Manager', 'neha.kapoor@company.com', '9000000003'),

(4, 'Finance Panel', 'Suresh Rao', 'Finance Manager', 'suresh.rao@company.com', '9000000004'),

(5, 'Marketing Panel', 'Kavya Shah', 'Marketing Manager', 'kavya.shah@company.com', '9000000005');

-- ========== -- INSERT VACANCY DATA -- ==========

INSERT INTO VACANCY (vacancy_id, title, description, status, created_on, department_id) VALUES

(1, 'Software Developer', 'Develop and maintain software applications', 'Open', '2026-08-01', 1),

(2, 'Data Analyst', 'Analyze business data and prepare reports', 'Open', '2026-08-03', 1),

(3, 'HR Executive', 'Handle recruitment and employee coordination', 'Open', '2026-08-05', 2),

(4, 'Financial Analyst', 'Prepare financial reports and perform analysis', 'Open', '2026-08-07', 3),

(5, 'Digital Marketing Executive', 'Manage digital marketing campaigns', 'Open', '2026-08-10', 4),

(6, 'Operations Executive', 'Manage operational processes and coordination', 'Closed', '2026-07-15', 5);

-- ========== -- INSERT APPLICATION DATA -- ==========

INSERT INTO APPLICATION (application_id, applied_on, status, cover_letter, resume_path, vacancy_id, candidate_id) VALUES

(1, '2026-08-04', 'Interview', 'I am interested in the Software Developer position.', '/resumes/aarav_sharma.pdf', 1, 1),

(2, '2026-08-05', 'Selected', 'I would like to apply for the Software Developer position.', '/resumes/ananya_reddy.pdf', 1, 2),

(3, '2026-08-06', 'Interview', 'I am applying for the Data Analyst position.', '/resumes/rohan_mehta.pdf', 2, 3),

(4, '2026-08-08', 'Selected', 'I am interested in joining the HR team.', '/resumes/priya_nair.pdf', 3, 4),

(5, '2026-08-09', 'Rejected', 'I would like to be considered for this position.', '/resumes/aditya_patnaik.pdf', 2, 5),

(6, '2026-08-11', 'Interview', 'I am applying for the Financial Analyst role.', '/resumes/sneha_iyer.pdf', 4, 6),

(7, '2026-08-12', 'Applied', 'I am interested in the Digital Marketing Executive role.', '/resumes/karan_verma.pdf', 5, 7),

(8, '2026-08-13', 'Selected', 'I am interested in the Operations Executive position.', '/resumes/meera_joshi.pdf', 6, 8),

(9, '2026-08-14', 'Interview', 'I am applying for the Software Developer role.', '/resumes/vivek_rao.pdf', 1, 9),

(10, '2026-08-15', 'Interview', 'I am interested in the Data Analyst position.', '/resumes/ishita_kapoor.pdf', 2, 10),

(11, '2026-08-16', 'Applied', 'I would like to apply for the HR Executive role.', '/resumes/rahul_das.pdf', 3, 11),

(12, '2026-08-17', 'Interview', 'I am applying for the Digital Marketing Executive role.', '/resumes/kavya_menon.pdf', 5, 12);

-- ========== -- INSERT QUALIFICATION DATA -- ==========

INSERT INTO QUALIFICATION (qualification_id, degree, institution, year_of_passing, percentage, candidate_id) VALUES

(1, 'B.Tech Computer Science', 'Osmania University', 2023, 82.50, 1),

(2, 'B.Tech Artificial Intelligence', 'Woxsen University', 2024, 86.20, 2),

(3, 'B.Sc Statistics', 'University of Mumbai', 2023, 79.80, 3),

(4, 'MBA Human Resources', 'Christ University', 2024, 84.10, 4),

(5, 'B.Tech Computer Science', 'KIIT University', 2024, 77.60, 5),

(6, 'MBA Finance', 'Symbiosis University', 2023, 88.30, 6),

(7, 'MBA Marketing', 'Amity University', 2024, 81.40, 7),

(8, 'BBA Operations', 'Manipal University', 2023, 83.70, 8),

(9, 'B.Tech Information Technology', 'JNTU Hyderabad', 2023, 80.40, 9),

(10, 'B.Sc Computer Science', 'Delhi University', 2024, 85.60, 10),

(11, 'MBA Human Resources', 'Xavier University', 2023, 82.90, 11),

(12, 'MBA Marketing', 'Amrita University', 2024, 87.10, 12);

-- ========== -- INSERT CANDIDATE_SKILL DATA -- ==========

INSERT INTO CANDIDATE_SKILL (candidate_id, skill_id, proficiency_level) VALUES

(1, 1, 'Advanced'), (1, 2, 'Intermediate'), (1, 3, 'Advanced'), (1, 7, 'Advanced'),

(2, 1, 'Advanced'), (2, 4, 'Advanced'), (2, 3, 'Advanced'), (2, 7, 'Advanced'),

(3, 3, 'Advanced'), (3, 5, 'Advanced'), (3, 1, 'Intermediate'),

(4, 7, 'Advanced'), (4, 8, 'Intermediate'),

(5, 1, 'Intermediate'), (5, 3, 'Intermediate'), (5, 4, 'Intermediate'),

(6, 3, 'Advanced'), (6, 5, 'Advanced'), (6, 10, 'Advanced'),

(7, 7, 'Advanced'), (7, 9, 'Advanced'), (7, 8, 'Intermediate'),

(8, 7, 'Advanced'), (8, 8, 'Advanced'),

(9, 1, 'Advanced'), (9, 3, 'Advanced'), (9, 6, 'Intermediate'),

(10, 3, 'Advanced'), (10, 5, 'Advanced'), (10, 7, 'Advanced'),

(11, 7, 'Advanced'), (11, 8, 'Intermediate'),

(12, 7, 'Advanced'), (12, 9, 'Advanced');

-- ========== -- INSERT INTERVIEW_ROUND DATA -- ==========

INSERT INTO INTERVIEW_ROUND (round_id, application_id, panel_id, round_no, round_type, status) VALUES

(1, 1, 1, 1, 'Technical', 'Completed'), (2, 1, 3, 2, 'HR', 'Completed'),

(3, 2, 1, 1, 'Technical', 'Completed'), (4, 2, 3, 2, 'HR', 'Completed'),

(5, 3, 2, 1, 'Technical', 'Completed'),

(6, 4, 3, 1, 'HR', 'Completed'),

(7, 6, 4, 1, 'Technical', 'Scheduled'),

(8, 7, 5, 1, 'Technical', 'Scheduled'),

(9, 9, 1, 1, 'Technical', 'Completed'), (10, 9, 3, 2, 'HR', 'Scheduled'),

(11, 10, 2, 1, 'Technical', 'Scheduled'),

(12, 12, 5, 1, 'Technical', 'Scheduled');

-- ========== -- INSERT SCHEDULE DATA -- ==========

INSERT INTO SCHEDULE (schedule_id, round_id, interview_datetime, mode, location, status) VALUES

(1, 1, '2026-08-20 10:00:00', 'Online', 'Microsoft Teams', 'Completed'),

(2, 2, '2026-08-22 11:00:00', 'In-Person', 'Hyderabad Office', 'Completed'),

(3, 3, '2026-08-21 14:00:00', 'Online', 'Google Meet', 'Completed'),

(4, 4, '2026-08-23 12:00:00', 'In-Person', 'Hyderabad Office', 'Completed'),

(5, 5, '2026-08-24 10:30:00', 'Online', 'Microsoft Teams', 'Completed'),

(6, 6, '2026-08-25 15:00:00', 'In-Person', 'Bengaluru Office', 'Completed'),

(7, 7, '2026-09-02 11:00:00', 'Online', 'Google Meet', 'Scheduled'),

(8, 8, '2026-09-03 14:30:00', 'In-Person', 'Delhi Office', 'Scheduled'),

(9, 9, '2026-08-28 10:00:00', 'Online', 'Microsoft Teams', 'Completed'),

(10, 10, '2026-09-04 11:30:00', 'Online', 'Google Meet', 'Scheduled'),

(11, 11, '2026-09-05 15:00:00', 'Online', 'Microsoft Teams', 'Scheduled'),

(12, 12, '2026-09-06 12:00:00', 'In-Person', 'Delhi Office', 'Scheduled');

-- ========== -- INSERT EVALUATION DATA -- ==========

INSERT INTO EVALUATION (evaluation_id, round_id, interviewer_id, score, remarks, result, evaluated_on) VALUES

(1, 1, 101, 82.50, 'Good programming and problem solving skills', 'Pass', '2026-08-20'),

(2, 2, 103, 85.00, 'Good communication and professional attitude', 'Pass', '2026-08-22'),

(3, 3, 102, 91.00, 'Excellent technical knowledge', 'Pass', '2026-08-21'),

(4, 4, 103, 88.00, 'Strong communication skills', 'Pass', '2026-08-23'),

(5, 5, 102, 76.50, 'Good analytical ability', 'Pass', '2026-08-24'),

(6, 6, 103, 89.00, 'Good HR knowledge and communication', 'Pass', '2026-08-25'),

(7, 9, 101, 84.00, 'Strong programming and database skills', 'Pass', '2026-08-28');

-- ========== -- INSERT OFFER DATA -- ==========

INSERT INTO OFFER (offer_id, application_id, offer_date, salary, valid_till, status) VALUES

(1, 2, '2026-08-26', 850000.00, '2026-09-05', 'Accepted'),

(2, 4, '2026-08-27', 650000.00, '2026-09-06', 'Accepted'),

(3, 9, '2026-08-30', 780000.00, '2026-09-10', 'Pending');

-- ========== -- INSERT JOINING DATA -- ==========

INSERT INTO JOINING (joining_id, offer_id, joining_date, employee_id, status) VALUES

(1, 1, '2026-09-15', 1001, 'Joined'),

(2, 2, '2026-09-20', 1002, 'Joined');

```
-- ========================================================= -- SHOW ALL TABLES -- =========================================================

SHOW TABLES;

-- ========================================================= -- VIEW DATA FROM ALL TABLES -- =========================================================

SELECT * FROM DEPARTMENT;

SELECT * FROM CANDIDATE;

SELECT * FROM SKILL;

SELECT * FROM INTERVIEW_PANEL;

SELECT * FROM VACANCY;

SELECT * FROM APPLICATION;

SELECT * FROM QUALIFICATION;

SELECT * FROM CANDIDATE_SKILL;

SELECT * FROM INTERVIEW_ROUND;

SELECT * FROM SCHEDULE;

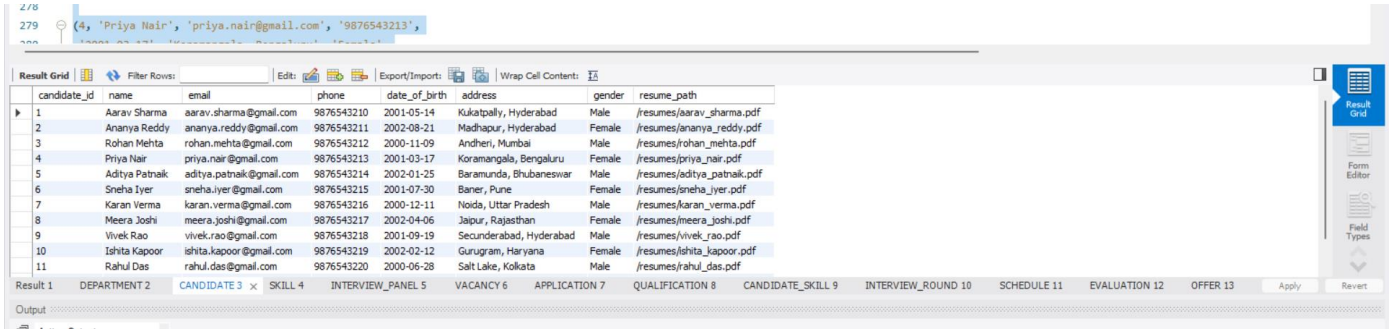
SELECT * FROM EVALUATION;

SELECT * FROM OFFER;

SELECT * FROM JOINING;
```

Appendix C: Additional Screenshots / Outputs

[ADDITIONAL DATABASE TABLE SCREENSHOT]



[ADDITIONAL WEBSITE SCREENSHOT]

